

CSS responsive web design 30m

Now we're going to look at how to implement responsive design. The basic idea of responsive design is to use HTML and CSS to allow the web content to be displayed in a way that matches the capabilities of the device being used. This makes use of some CSS that allows the styling to be adjusted according to the size of the screen. You can test this by dragging the size of the browser to resize this page and watch the elements shuffle accordingly.

Adopting responsive design allows a developer to provide a site that works well on both desktop and mobile devices. It avoids the problem of having to create specific desktop and mobile versions of an application or website. This saves time and effort, as well as ensuring consistency between the sites.

You may also find people use your application in a way you didn't expect. By allowing users choice in how they access/use your web application, you reduce potential frustrations that could drive them to a competing application or service.

A great example of responsive design is Google Maps. Google Maps can be used on a desktop or tablet, but it also works well on smaller mobile devices. So, you can use it to plan a journey on your home device or use it on your mobile once you are out and about (and possibly lost).

Media queries

Media queries are a key feature in implementing responsive web design using CSS. Media queries are used to modify the page layout as the device size changes.

[Skip to main content](#)

With media queries, you can define completely different styles and layouts for each different device size. For example, you may want to add breakpoints: points where the layout will behave differently depending on the size of the device (W3Schools, n.d.).^{fnref:dfc1d9}



Breakpoints in responsive web design enable websites to adapt dynamically to different screen sizes and device types, optimising the user experience. As seen in the image, the layout changes based on the device: on smartphones, content is stacked vertically with simplified menus to suit narrow screens; tablets offer slightly expanded layouts with some side-by-side elements, leveraging the medium screen size; laptops fully utilize the wider screen space, displaying multi-column layouts, detailed content, and expanded navigation. By using CSS media

[Skip to main content](#)

queries to define specific screen width thresholds, breakpoints ensure that websites remain accessible, readable, and user-friendly across all devices.

© © Getty Images



A media query consists of one or more media feature expressions, which are combined using logical operators and a media type. In a media query, you write a set of CSS rules that will only be applied if a condition is met. The condition usually refers to the width of the screen or viewport.

For example, if the media type matches the device used to display the page and the expression computes true, then the rule is applied.

A media query uses an @media rule to define CSS rules inside the curly brackets.

For example, if the viewport is less than 400 pixels wide, we want the font to be larger and the images to be the full width of the page. So, we might write:



[Skip to main content](#)



```
1 @media only screen and (max-width: 400px) {  
2     h1 { font-size:2em; }  
3     img { width:100%; }  
4 }
```



Note how the two declaration blocks are wrapped inside the media query, and the media query only applies up to a maximum width of 400 pixels.

In a previous topic, we used the Responsive Image Breakpoints Generator tool to create several images at different sizes. These various images would be selected based on the media queries.

Example 6.19 below uses the `@media` rule to add several device breakpoints to ensure the content is responsive to differences in screen size.

Have a go at changing your screen size to see how the text properties change as the screen size changes.



[Skip to main content](#)

HTML

CSS

Result

EDIT ON

LIVE

```
/* Extra small devices (phones,
600px and down) */
@media only screen and (max-width:
600px) {
  .row h2 {
    color: yellow;
    font-size: 10px;
  }
}

/* Small devices (portrait tablets
and large phones, 600px and up) */
@media only screen and (min-width:
600px) {
  .row h2 {
    color: blue;
    font-size: 14px;
  }
}

/* Medium devices (landscape
tablets, 768px and up) */
@media only screen and (min-width:
768px) {
  .row h2 {
    color: green;
    font-size: 18px;
  }
}
```

Resources1x0.5x0.25xRerun

Example 6.19 Responsive design



☒ Breakpoints

Non-Assessed

[Skip to main content](#)

In this activity, you will use media queries to create four common device breakpoints.

Use the code below to create device breakpoints for a:

- small device
- medium device
- large device
- printer.

You may choose how you'll format each breakpoint. We've chosen to change the background colour for different screen sizes.

```
/* Small devices (portrait tablets and large phones, 600px and up) */
@media only screen and (min-width: 600px) {...}

/* Medium devices (landscape tablets, 768px and up) */
@media only screen and (min-width: 768px) {...}

/* Large devices (laptops/desktops, 992px and up) */
@media only screen and (min-width: 992px) {...}]

/* Print – for all printers*/
@media print {header, footer, aside, form, ... { }
```

☐ I have successfully completed this task!

Submit



Next, let's explore CSS animations, which bring websites to life by adding movement and transitions to elements.



[Skip to main content](#)

Thoughts on this page?

